Django Complete Notes A–Z
Django 6.0 | Beginner to Advanced | English + বাংলা ব্যাখ্যা

Purpose: Complete learning notes, practical examples, interview preparation, project development, and production reference.

Technology: Python, Django, Django ORM, Templates, Forms, Authentication, REST APIs, Testing, Security, Deployment.

Official documentation: https://docs.djangoproject.com/en/6.0/

Version compatibility: Django 6.0 supports Python 3.12, 3.13, and 3.14.

> **Important:** These notes use educational examples. Adjust project names, database settings, credentials, and business logic to match your own project. Never commit production secrets to source control.

Table of Contents — Parts 1–49

1. Introduction to Django
2. Web Development Fundamentals
3. Python Prerequisites for Django
4. Environment Setup and Installation
5. Creating Projects and Applications
6. Django Project Structure
7. Settings and Configuration
8. URLs and URL Routing
9. Views and HTTP Requests
10. HttpResponse, Redirects, and Status Codes
11. Django Templates
12. Template Inheritance and Reusable Components
13. Static Files: CSS, JavaScript, and Images
14. Models and Database Fields
15. Database Configuration and Migrations
16. Django ORM Fundamentals
17. CRUD Operations
18. QuerySets, Filtering, and Lookups

19. Relationships and Related Objects
20. Query Optimization and Database Indexes
21. Forms and ModelForms
22. Form Validation and Error Handling
23. Django Admin
24. Authentication and User Management
25. Authorization, Groups, and Permissions
26. Sessions, Cookies, and Messages
27. File and Image Uploads
28. Class-Based Views
29. Middleware and Request Processing
30. Signals and Application Configuration
31. Email and Notifications
32. Pagination, Search, and Filtering
33. Django REST Framework Fundamentals
34. Serializers and API Validation
35. API Views, ViewSets, and Routers
36. API Authentication and Permissions
37. API Pagination, Filtering, and Versioning
38. Testing and Debugging
39. Logging and Error Monitoring
40. Security Best Practices
41. Transactions and Advanced ORM
42. Caching and Performance Optimization
43. Background Tasks and Scheduled Work
44. Async Views and Asynchronous Programming
45. Internationalization and Localization
46. Deployment and Production Configuration
47. Docker, Environment Variables, and CI/CD
48. Advanced Architecture and Design Patterns
49. Interview Preparation, Cheat Sheets, and Learning Roadmap

---

PART 1: Introduction to Django

1.1 What Is Django?

Django is a high-level Python web framework used to build secure, maintainable, and scalable web applications quickly.

It provides built-in tools for:

- URL routing
- Request and response handling
- Database operations through the ORM
- HTML template rendering
- Forms and validation
- Authentication and permissions
- Administrative interfaces
- Security protections
- Testing and debugging

বাংলা ব্যাখ্যা: Django হলো Python দিয়ে ওয়েবসাইট ও ওয়েব অ্যাপ্লিকেশন তৈরি করার একটি framework। এটি অনেক প্রয়োজনীয় কাজ আগে থেকেই তৈরি করে দেয়। ফলে প্রতিটি কাজ শূন্য থেকে লিখতে হয় না।

For example, Django can help you build:

- A blog website
- An e-commerce application
- A school management system
- A job portal
- A REST API backend
- A customer management system
- A booking application

1.2 What Is a Web Framework?

A web framework provides reusable components and conventions for building web applications.

Without a framework, a developer may need to implement request handling, URL matching, database access, form validation, authentication, and many other features manually.

A framework organizes these responsibilities into a predictable structure.

বাংলা: Framework হলো এমন একটি কাঠামো, যা নির্দিষ্ট নিয়মে application তৈরি করতে সাহায্য করে। এটি development দ্রুত করে এবং code maintain করা সহজ করে।

1.3 How Does Django Work?

A simplified Django request-response flow:

1. A user opens a URL in the browser.
2. The browser sends an HTTP request.
3. Django receives the request.
4. URL routing selects the appropriate view.
5. The view executes application logic.
6. The view may query the database using the ORM.
7. Django returns an HTTP response, HTML page, redirect, or JSON data.
8. The browser displays or processes the response.

Example:

```text
Browser
   |
   | HTTP Request: GET /students/
   v
Django URL Router
   |
   v
View Function
   |
   v
Django ORM ---> Database
   |
   v
Template Rendering
   |
   v
HTTP Response
   |
   v
Browser
```

বাংলা ব্যাখ্যা: Browser থেকে request এলে Django URL দেখে ঠিক করে কোন view চলবে। View প্রয়োজন হলে database থেকে তথ্য আনে এবং response তৈরি করে browser-এ পাঠায়।

1.4 What Is MVT Architecture?

Django commonly follows the **MVT pattern**:

- **Model:** Defines data structure and database-related behavior.
- **View:** Processes requests, executes application logic, and returns responses.
- **Template:** Defines the presentation, usually HTML.

Component	Responsibility	বাংলা ব্যাখ্যা
Model	Data and database behavior	তথ্য ও database-এর কাঠামো
View	Request handling and business logic	request প্রক্রিয়াকরণ ও application logic
Template	Presentation and HTML rendering	ব্যবহারকারীর কাছে তথ্য দেখানো

Example:

Model

```python
from django.db import models


class Student(models.Model):
    name = models.CharField(max_length=100)
    email = models.EmailField(unique=True)

    def __str__(self):
        return self.name
```

View

```python
from django.shortcuts import render
```

```
from .models import Student



def student_list(request):
    students = Student.objects.all()

    return render(
        request,
        "students/student_list.html",
        {"students": students},
    )
```

Template

```html
<h1>Student List</h1>

<ul>
    {% for student in students %}
        <li>{{ student.name }} — {{ student.email }}</li>
    {% empty %}
        <li>No students found.</li>
    {% endfor %}
</ul>
```

Here, the model defines student data, the view retrieves records, and the template displays them.

বাংলা: Model তথ্যের কাঠামো তৈরি করে, View প্রয়োজনীয় কাজ সম্পন্ন করে, আর Template browser-এ তথ্য উপস্থাপন করে।

1.5 Important Django Features

Built-in ORM

Django ORM lets you query databases using Python expressions.

```python
Student.objects.filter(name__icontains="rahim")
```

This retrieves students whose names contain "rahim", case-insensitively, subject to the database backend's behavior.

Authentication

Django includes user authentication, password hashing, login utilities, and permission mechanisms.

Django Admin

The admin interface provides a convenient way for authorized staff to manage application data.

Forms and Validation

Django forms help validate user input on the server.

Security

Django includes protections and tools for common web security concerns. Correct configuration and safe application code are still required.

Reusable Applications

Django applications can separate features into modules, making projects easier to maintain.

1.6 Django Advantages and Limitations

Advantages

- Rapid development
- Built-in authentication and admin

- ORM and migration support
- Structured project organization
- Security-oriented defaults
- Extensive documentation and ecosystem
- Suitable for many data-driven applications

Limitations

- Some small applications may not need a full framework.
- Developers must understand the framework's conventions.
- Poor ORM usage can cause slow database queries.
- Production deployment requires additional infrastructure and configuration.
- Third-party packages may not immediately support every Django release.

1.7 Django vs Flask vs FastAPI

Feature	Django	Flask	FastAPI
Main purpose	Full-featured web framework	Lightweight web framework	API-focused framework
Built-in ORM	Yes	No	No built-in ORM
Built-in admin	Yes	No	No built-in admin
Authentication tools	Built-in foundation	Extensions commonly used	Additional libraries commonly used
Automatic API schema	Additional API tooling may be used	Extensions available	Strong built-in support
Best fit	Full web applications and data-driven systems	Lightweight, flexible applications	APIs and typed asynchronous endpoints

No framework is automatically best for every project. Choose based on requirements, team expertise, integrations, and maintainability.

1.8 When Should You Use Django?

Django is a strong option when you need:

- A relational database-backed application
- Authentication and permissions

- An administrative interface
- Forms and server-rendered pages
- A maintainable Python backend
- APIs with Django REST Framework or other API tools

1.9 Interview Questions

Q1. What is Django?

Django is a high-level Python web framework that provides reusable components for building secure and maintainable web applications.

Q2. What is MVT?

MVT stands for Model-View-Template. The Model handles data, the View processes requests and application logic, and the Template handles presentation.

Q3. Is Django suitable for REST APIs?

Yes. Django can be used to build REST APIs, commonly with Django REST Framework.

Q4. Does Django automatically make an application secure?

No. Django provides security features, but developers must configure settings correctly, validate inputs, enforce authorization, manage secrets, and follow secure coding practices.

PART 2: Web Development Fundamentals

2.1 What Is a Website?

A website is a collection of web pages and resources accessible through a browser.

A web application is a website or online system that provides interactive functionality, such as registration, ordering products, managing records, or submitting forms.

বাংলা: সাধারণ website তথ্য দেখাতে পারে; web application সাধারণত ব্যবহারকারীর input নেয়, তথ্য প্রক্রিয়া করে এবং প্রয়োজন অনুযায়ী database-এর সঙ্গে কাজ করে।

2.2 Frontend and Backend

Frontend

The frontend is the part of an application that users interact with.

Common technologies:

- HTML – page structure
- CSS – visual styling
- JavaScript – browser-side interaction

Backend

The backend processes requests, applies business rules, manages data, and communicates with external services.

Common backend technologies include Python with Django, Java with Spring, JavaScript with Node.js, and others.

বাংলা: Frontend ব্যবহারকারীর সামনে দেখা যায়। Backend server-side logic, authentication, database operations এবং business rules পরিচালনা করে।

2.3 Client-Server Architecture

A client sends a request to a server. The server processes the request and returns a response.

```text
Client (Browser)
        |
        | HTTP Request
```

```
       v
Web Server / Django Application
       |
       | Database Query, if required
       v
Database
       |
       v
Django Response
       |
       v
Client (Browser)
```

A browser is one type of client. Mobile applications and other services can also act as clients.

2.4 What Is HTTP?

HTTP stands for **Hypertext Transfer Protocol**. It defines how clients and servers exchange requests and responses.

Common HTTP methods:

Method	Typical purpose
GET	Retrieve data
POST	Submit data or trigger a non-idempotent operation
PUT	Replace a resource representation
PATCH	Partially update a resource
DELETE	Delete a resource

The correct method depends on the endpoint's contract. HTTP semantics and application behavior should remain consistent.

Example: GET

```http
GET /students/
```

This commonly requests a list of students.

Example: POST

```http
POST /students/
Content-Type: application/json

{
    "name": "Rahim",
    "email": "rahim@example.com"
}
```

This could create a student in an API that defines `POST /students/` as a creation endpoint.

বাংলা: GET সাধারণত তথ্য পড়ার জন্য, POST তথ্য পাঠানো বা নতুন resource তৈরির জন্য ব্যবহৃত হয়। বাস্তবে কোন endpoint কী করবে, তা API design অনুযায়ী নির্ধারিত হয়।

2.5 Common HTTP Status Codes

Code	Meaning
200	OK
201	Created
204	No Content
301	Moved Permanently
302	Found / Temporary Redirect
400	Bad Request
401	Unauthorized / Authentication required in common API usage
403	Forbidden
404	Not Found
405	Method Not Allowed
409	Conflict
422	Unprocessable Content
429	Too Many Requests
500	Internal Server Error
503	Service Unavailable

The precise status code should reflect the endpoint's behavior and protocol requirements.

2.6 What Is a URL?

URL stands for **Uniform Resource Locator**.

Example:

```text
https://example.com/students/?page=2
```

Components:

- `https` — scheme
- `example.com` — host
- `/students/` — path
- `page=2` — query parameter

বাংলা: URL হলো web resource-এর ঠিকানা। Path দিয়ে resource বা endpoint চেনা যায়, আর query parameter দিয়ে অতিরিক্ত filter বা pagination information পাঠানো যায়।

2.7 What Is JSON?

JSON stands for **JavaScript Object Notation**. It is a common format for exchanging structured data.

Example:

```json
{
    "id": 1,
    "name": "Rahim",
    "email": "rahim@example.com",
    "is_active": true
}
```

```

JSON supports objects, arrays, strings, numbers, booleans, and `null`.

## 2.8 What Is an API?

An API (Application Programming Interface) defines how one software component communicates with another.

A REST-style API commonly exposes resources through URLs and uses HTTP methods to retrieve or modify them.

Example endpoints:

```text
GET     /api/students/
GET     /api/students/1/
POST    /api/students/
PATCH   /api/students/1/
DELETE  /api/students/1/
```

These are illustrative endpoints. A real API must implement and document its own resource and permission rules.

2.9 What Is a Database?

A database stores and organizes information so that applications can retrieve and modify it.

Popular relational databases include:

- PostgreSQL
- MySQL
- SQLite
- Oracle
```

A database can contain tables, rows, columns, relationships, constraints, and indexes.

2.10 What Is SQL?

SQL stands for **Structured Query Language**. It is commonly used to query and modify relational database data.

Example:

```sql
SELECT *
FROM students
WHERE age >= 18;
```

Django ORM equivalent, assuming a suitable `Student` model with an `age` field:

```python
Student.objects.filter(age__gte=18)
```

Understanding SQL remains important even when using an ORM.

2.11 What Is Authentication vs Authorization?

Authentication: Determines who a user is.

Authorization: Determines what an authenticated user is allowed to do.

Example:

- Login verifies a user's identity.
- A permission check determines whether the user may delete a record.

বাংলা: Authentication মানে পরিচয় যাচাই। Authorization মানে সেই ব্যবহারকারীর কোন কাজ করার অনুমতি আছে তা যাচাই।

2.12 Interview Questions

Q1. What is the difference between frontend and backend?

Frontend handles user-facing interaction; backend handles server-side processing, business logic, and data operations.

Q2. What is HTTP?

HTTP is a protocol used for communication between web clients and servers.

Q3. What is JSON?

JSON is a lightweight data-interchange format commonly used by APIs.

Q4. What is the difference between authentication and authorization?

Authentication verifies identity; authorization determines access rights.

PART 3: Python Prerequisites for Django

3.1 Why Learn Python Before Django?

Django is built with Python. You should understand Python's syntax, functions, classes, modules, exceptions, and data structures before building larger Django applications.

বাংলা: Django শেখার আগে Python-এর মৌলিক বিষয়গুলো জানা প্রয়োজন। এতে Django-এর model, view, form এবং class-based view বোঝা সহজ হয়।

3.2 Variables and Data Types

```python
name = "Rahim"
age = 22
price = 199.50
is_active = True
```

Common data types:

- `str` — text
- `int` — integer
- `float` — floating-point number
- `bool` — Boolean value
- `list` — ordered mutable collection
- `tuple` — ordered immutable collection
- `dict` — key-value mapping
- `set` — collection of unique values

3.3 Lists, Dictionaries, and Loops

```python
students = ["Rahim", "Karim", "Amina"]

for student in students:
    print(student)
```

Dictionary example:

```python
student = {
    "name": "Rahim",
    "age": 22,
    "email": "rahim@example.com",
}

print(student["name"])
```

Django frequently uses dictionaries to pass data into templates:

```python
context = {
    "title": "Student List",
    "students": students,
}
```

3.4 Conditional Statements

```python
age = 20

if age >= 18:
    print("Adult")
else:
    print("Minor")
```

Conditional logic is common in views, forms, permissions, and business rules.

3.5 Functions

A function groups reusable logic.

```python
def calculate_total(price, quantity):
    return price * quantity



total = calculate_total(100, 3)
print(total)
```

In Django, function-based views are Python functions that receive a request and return a response.

3.6 Modules and Imports

A module is a Python file containing reusable definitions.

```python
from datetime import date



today = date.today()
print(today)
```

Django projects contain multiple Python modules, including `models.py`, `views.py`, `forms.py`, and `urls.py`.

3.7 Classes and Objects

A class defines the structure and behavior of objects.

```python
class Student:
    def __init__(self, name, email):
        self.name = name
        self.email = email

    def display_name(self):
        return self.name



student = Student("Rahim", "rahim@example.com")

print(student.display_name())
```

Django models are classes that inherit from `django.db.models.Model`.

3.8 Inheritance

Inheritance allows a class to reuse or extend behavior from another class.

```python
class Person:
    def __init__(self, name):
        self.name = name



class Student(Person):
    def study(self):
        return f"{self.name} is studying."



student = Student("Rahim")
print(student.study())
```

Django uses inheritance in models, class-based views, forms, and other framework components.

3.9 Exceptions and Error Handling

```python
try:
    number = int("123")
except ValueError:
    print("Invalid integer")
else:
    print(number)
```

Handle expected errors appropriately. Avoid silently swallowing exceptions or exposing sensitive debugging details to users.

3.10 List Comprehensions

```python
numbers = [1, 2, 3, 4, 5]
```

```
squares = [number ** 2 for number in numbers]

print(squares)
```

List comprehensions are useful, but avoid building huge lists unnecessarily when a generator or database query would be more efficient.

3.11 Virtual Environments

A virtual environment isolates Python dependencies for a project.

```bash
python -m venv .venv
```

Activate on Windows Command Prompt:

```bat
.venv\Scripts\activate.bat
```

Activate on Windows PowerShell:

```powershell
.\.venv\Scripts\Activate.ps1
```

Activate on macOS or Linux:

```bash
source .venv/bin/activate
```

3.12 Install Django 6.0

Confirm that the active Python version is supported:

```bash
python --version
```

Install Django 6.0 in the activated virtual environment:

```bash
python -m pip install "Django>=6.0,<6.1"
```

Verify the installed version:

```bash
python -m django --version
```

For a reproducible project, record the exact tested dependencies in a requirements or lock file. A version range is useful for installation, but it is not a substitute for dependency pinning in production.

3.13 Recommended Python Topics Before Continuing

- Variables and data types
- Operators and conditionals
- Loops and comprehensions
- Functions and arguments
- Lists, tuples, dictionaries, and sets
- Classes and inheritance
- Modules and packages
- Exceptions
- File handling
- Virtual environments
- `pip` and dependency management
- Basic SQL and HTTP concepts

3.14 Practice Exercises

1. Write a function that calculates a product's total price.

2. Create a list of students and print each student's name.
3. Create a dictionary representing a student.
4. Write a class with two attributes and one method.
5. Handle a `ValueError` when converting text to an integer.
6. Create and activate a virtual environment.
7. Install Django 6.0 and verify the version.

3.15 Interview Questions

Q1. What is a Python class?

A class is a blueprint for creating objects that combine data and behavior.

Q2. What is inheritance?

Inheritance allows a class to reuse or extend functionality from another class.

Q3. Why use a virtual environment?

It isolates project dependencies and reduces conflicts between projects.

Q4. What is the difference between a list and a tuple?

Lists are mutable; tuples are immutable.

PART 4: Environment Setup and Installation

4.1 Requirements

Before starting, install:

- Python 3.12, 3.13, or 3.14
- A terminal or command prompt
- A code editor such as VS Code

- Git (recommended)
- A supported database, if not using SQLite

Django includes SQLite support for development. For production, choose a database based on application requirements and deployment infrastructure.

4.2 Create a Project Directory

```bash
mkdir django_project
cd django_project
```

4.3 Create a Virtual Environment

```bash
python -m venv .venv
```

Activate it using the command for your operating system.

4.4 Install Django

```bash
python -m pip install "Django>=6.0,<6.1"
```

Check the version:

```bash
python -m django --version
```

4.5 Create a Django Project

```bash
django-admin startproject config .
```

The trailing dot tells Django to create the project in the current directory.

4.6 Run the Initial Migrations

```bash
python manage.py migrate
```

This creates the database tables required by Django's installed applications and migration system.

4.7 Start the Development Server

```bash
python manage.py runserver
```

Open the local address printed by the command, commonly:

```text
http://127.0.0.1:8000/
```

The development server is intended for development and testing, not production deployment.

4.8 Check the Project

```bash
python manage.py check
```

This runs Django's system checks. It does not prove that every feature works or that the application is production-ready.

4.9 Create an Application

```bash
python manage.py startapp students
```

An application is a reusable module that groups related functionality.

4.10 Register the Application

Open `config/settings.py` and add the application to `INSTALLED_APPS`:

```python
INSTALLED_APPS = [
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
    "django.contrib.staticfiles",

    "students",
]
```

Register an application before relying on its models, migrations, templates, or other app integrations.

4.11 Create an Initial View

In `students/views.py`:

```python
from django.http import HttpResponse



def home(request):
    return HttpResponse("Welcome to Django!")
```

4.12 Configure Application URLs

Create `students/urls.py`:

```python
from django.urls import path

from . import views



urlpatterns = [
    path("", views.home, name="home"),
]
```

Then edit `config/urls.py`:

```python
from django.contrib import admin
from django.urls import include, path



urlpatterns = [
    path("admin/", admin.site.urls),
    path("", include("students.urls")),
]
```

Start the server:

```bash
python manage.py runserver
```

Visit `http://127.0.0.1:8000/`. You should see the text returned by the view.

4.13 Create a Superuser

```bash
python manage.py createsuperuser
```

Follow the prompts to create an administrative user.

Open `/admin/` on the local development server and sign in.

4.14 Essential Django Commands

```bash
# Create a project
django-admin startproject config .

# Create an app
python manage.py startapp students

# Apply migrations
python manage.py migrate

# Create migrations from model changes
python manage.py makemigrations

# Run the development server
python manage.py runserver

# Create an administrative user
python manage.py createsuperuser

# Open the Django shell
python manage.py shell

# Run system checks
python manage.py check

# Run tests
python manage.py test
```

4.15 Common Setup Problems

Problem: `django-admin` is not recognized.

Possible causes:

- The virtual environment is not activated.
- Django is installed in a different Python environment.
- The command-line executable is not on the active PATH.

Try:

```bash
python -m django --version
python -m pip show Django
```

Problem: `ModuleNotFoundError`.

Check the active interpreter and install the required package into that environment.

Problem: Database tables are missing.

Run:

```bash
python manage.py migrate
```

If you have changed models, first generate migrations:

```bash
python manage.py makemigrations
python manage.py migrate
```

Problem: Port 8000 is already in use.

Start the development server on another port:

```bash
python manage.py runserver 8001
```

4.16 Interview Questions

Q1. What is a virtual environment?

It is an isolated Python environment used to manage project-specific dependencies.

Q2. What is `manage.py`?

It is a command-line utility for administrative tasks in a Django project.

Q3. What does `migrate` do?

It applies database migrations to bring the database schema in line with migration history.

Q4. Is `runserver` suitable for production?

No. It is Django's development server. Production deployments should use an appropriate production server and deployment architecture.

PART 5: Creating Projects and Applications

5.1 Project vs Application

A **Django project** is the configuration and collection of applications that together make up a website or service.

A **Django application** is a module that provides a particular feature or group of related features.

Example:

```text
E-commerce Project
├── config
├── accounts
├── products
├── cart
├── orders
└── payments
```

বাংলা: Project হলো সম্পূর্ণ website বা service-এর মূল কাঠামো। App হলো তার নির্দিষ্ট feature-এর module। যেমন `accounts` app user management করতে পারে, আর `products` app product management করতে পারে।

5.2 Create a Project

```bash
django-admin startproject config .
```

Typical structure:

```text
django_project/
├── manage.py
├── config/
│   ├── __init__.py
│   ├── asgi.py
│   ├── settings.py
│   ├── urls.py
│   └── wsgi.py
└── .venv/
```

The virtual environment directory is not part of Django's source code and should not be committed to version control.

5.3 Important Project Files

`manage.py`

Runs Django management commands for the project.

`settings.py`

Contains project configuration, such as installed applications, middleware, database settings, templates, static files, and security settings.

`urls.py`

Defines URL patterns that route incoming requests to views.

`asgi.py`

Provides the ASGI application entry point, useful for compatible asynchronous server deployments.

`wsgi.py`

Provides the WSGI application entry point for compatible server deployments.

`__init__.py`

Marks the directory as a Python package.

5.4 Create an Application

```bash
python manage.py startapp students
```

Typical application structure:

```text
students/
├── __init__.py
├── admin.py
├── apps.py
├── migrations/
│   └── __init__.py
├── models.py
├── tests.py
└── views.py
```

You can add `urls.py`, `forms.py`, `services.py`, `selectors.py`, and other modules as the application grows.

5.5 Register the Application

Add the application to `INSTALLED_APPS` in `config/settings.py`:

```python
INSTALLED_APPS = [
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
    "django.contrib.staticfiles",

    "students.apps.StudentsConfig",
]
```

Using the application configuration class makes the app's configuration explicit.

5.6 Application Configuration

In `students/apps.py`:

```python
from django.apps import AppConfig


class StudentsConfig(AppConfig):
    default_auto_field = "django.db.models.BigAutoField"
    name = "students"
```

The `name` attribute identifies the Python package containing the application.

5.7 Add a View

In `students/views.py`:

```python
from django.http import HttpResponse


def student_home(request):
    return HttpResponse("Students application is working.")
```

5.8 Add Application URLs

Create `students/urls.py`:

```python
from django.urls import path

from . import views


app_name = "students"

urlpatterns = [
    path("", views.student_home, name="home"),
]
```

```

The `app_name` creates a namespace for reversing named URLs.

## 5.9 Include Application URLs in the Project

In `config/urls.py`:

```python
from django.contrib import admin
from django.urls import include, path


urlpatterns = [
    path("admin/", admin.site.urls),
    path("students/", include("students.urls")),
]
```

Now the view is available at:

```text
http://127.0.0.1:8000/students/
```

Because the app uses the `students` namespace, you can reverse its URL in Python:

```python
from django.urls import reverse


url = reverse("students:home")
```

In a template:

```html
<a href="{% url 'students:home' %}">Students Home</a>
```
```

```

## 5.10 Recommended Application Organization

Keep related functionality together, but avoid splitting every tiny function into a separate app.

Example:

```text
config/
├── settings.py
├── urls.py
├── asgi.py
└── wsgi.py

accounts/
├── models.py
├── views.py
├── forms.py
├── urls.py
└── tests.py

students/
├── models.py
├── views.py
├── forms.py
├── urls.py
└── tests.py

templates/
static/
manage.py
```

For larger projects, use packages within an app to organize complex views, models, services, and tests.

5.11 Common Mistakes
```

- Creating an app but not adding it to `INSTALLED_APPS`.
- Defining a view but forgetting to add a URL pattern.
- Including an incorrect URL module.
- Using inconsistent URL namespaces.
- Duplicating the same business logic across multiple views.
- Storing secrets in committed settings files.
- Treating development configuration as production configuration.

5.12 Practice Task

Create a project called `school_project` and an application called `students`.

Requirements:

1. Register the `students` app.
2. Create a view returning `Welcome to Students App`.
3. Define a URL named `home`.
4. Include the application URLs in the project.
5. Open the page in the browser.
6. Reverse the named URL in a template.

5.13 Interview Questions

Q1. What is the difference between a project and an app?

A project contains the overall configuration and one or more apps. An app groups related functionality.

Q2. Why is `INSTALLED_APPS` important?

It tells Django which applications to load and integrate into the project.

Q3. Why use `include()` in URL configuration?

It allows URL patterns to be organized into reusable modules.

Q4. What is URL namespacing?

It distinguishes URL names belonging to different applications, helping prevent naming conflicts.

End of Initial Section

Completed in this installment: **Parts 1–5 of 49**.

Next sections will continue in order with **Part 6: Django Project Structure**, followed by settings, URL routing, views, templates, models, ORM, forms, authentication, REST APIs, testing, security, performance, deployment, and advanced topics.

Official reference: https://docs.djangoproject.com/en/6.0/